Denominación	Taller de Pruebas de Software – App de usuarios (API y web) con Vitest, Supertest, Playwright y TestSprite
Nombre y código del Programa de formación:	TO Análisis y Desarrollo de Software
Competencia:	Controlar La Calidad Del Servicio De Software De Acuerdo Con Los Estándares Técnicos
Identificación de la actividad:	Aplicar técnicas y herramientas de pruebas automatizadas (unitarias, de integración, end-to-end y asistidas por IA) para verificar la API y la web de la App de usuarios, y documentar los resultados con evidencias y reportes de cobertura.

Taller de Pruebas de Software – App de usuarios (API y web)

Identificación de la actividad: Diseñar, automatizar y ejecutar pruebas sobre el proyecto App de usuarios, compuesto por una API REST (Node.js, Express, MongoDB y JWT) y una aplicación web (Vue 3, Vite y Vue Router). Se aplican pruebas unitarias y de integración con Vitest y Supertest, pruebas de componentes con Vue Test Utils, pruebas end-to-end con Playwright y pruebas generadas con inteligencia artificial con TestSprite, y se miden con reportes de cobertura (coverage).

Instrucciones: Leer con atención la actividad propuesta por el instructor, desarrollar cada uno de los puntos propuestos y presentarla de forma digital. El taller se trabaja en parejas; al terminar cada punto se debe hacer un commit con el avance. Todas las evidencias (reportes, capturas y archivos generados) se cargan en el repositorio y en Classroom según la tabla de evidencias de este documento.

Para la realización de cada uno de los ejercicios es necesario seguir el siguiente procedimiento:

1. Leer completamente el enunciado de cada ejercicio hasta comprender con claridad lo que se está pidiendo.
2. Descomprimir app-usuarios.rar, leer el código de api/ y web/ e identificar qué herramienta corresponde a cada caso de prueba (Vitest, Supertest, Vue Test Utils, Playwright o TestSprite).
3. Preparar el entorno: crear el repositorio en GitHub con las carpetas api/ y web/, crear api/.env a partir de .env.example, ejecutar npm install en cada carpeta, cargar los datos con npm run seed y comprobar que la API (puerto 3000) y la web (puerto 5173) funcionan.
4. Escribir las pruebas de cada punto con nombres descriptivos (qué se prueba y qué se espera).

5. Ejecutar las pruebas con los scripts de npm y corregir el código o la prueba cuando algo falle.
6. Generar el reporte de cobertura y verificar que se cumplen los umbrales mínimos.
7. Si está correcto continuar con el siguiente ejercicio, si no volver al paso 4.
8. Guardar las evidencias en la carpeta evidencias/ y hacer commit al terminar cada punto.

Descripción del taller

La App de usuarios ya está construida: la API permite registrarse, iniciar sesión, consultar y editar el perfil y cambiar la contraseña, y los administradores tienen un CRUD de usuarios con búsqueda, filtro por rol, paginación y borrado lógico. La web consume esa API con Vue Router, guarda el token en localStorage y protege las rutas según la sesión y el rol. Antes de publicarla, el líder técnico exige que ambas partes tengan pruebas automatizadas que se ejecuten con un solo comando, un reporte de cobertura que demuestre qué parte del código está probada y una revisión adicional con TestSprite, un agente de pruebas con inteligencia artificial que genera y ejecuta casos de prueba a partir de los requisitos del proyecto.

Partes del proyecto que se van a probar:

Parte	Carpetas y archivos	Qué contiene
API	models/usuario.js	Schema, hash con bcrypt en pre("save"), validarPassword() y toJSON() sin password y con uid
API	routes/ y helpers/reglas-usuario.js	Validaciones con express-validator (nombre, email, contraseña, rol, paginación)
API	controllers/auth.js y usuarios.js	Login, registro, perfil, cambio de contraseña y CRUD con reglas de negocio
API	middlewares/	validarCampos (400), validarJWT (401) y validarRol (403)
Web	src/utils/validators.js	Validaciones del lado del cliente (las mismas reglas de la API)
Web	src/services/http.js	request(): agrega x-token, convierte los errores en ApiError y avisa los 401
Web	src/composables/	useAuth (sesión y rol) y useToast (notificaciones)
Web	src/components/ y src/views/	FormField, PaginationBar, ConfirmDialog, AlertMessage y las vistas, con atributos data-test
Web	src/router/index.js	Guardas: requiresAuth, guestOnly y requiresAdmin

El taller usa las siguientes herramientas:

Herramienta	Tipo de prueba	Se usa en
Vitest	Unitarias e integración; ejecutor y cobertura	API y web

Supertest	Integración HTTP	API: peticiones a los endpoints sin levantar el servidor
mongodb-memory-server	Base de datos de prueba	API: MongoDB en memoria, sin tocar la base real
Vue Test Utils + jsdom	Componentes y composables	Web: montar componentes y simular al usuario
Playwright	End-to-end (E2E)	Web + API: flujos completos en un navegador real
@vitest/coverage-v8	Cobertura de código	API y web: reporte en consola y en HTML
TestSprite (MCP)	Pruebas generadas con IA	API y web: plan, ejecución y reporte automático

Requisitos técnicos generales

- Node.js 22 o superior. Cada carpeta tiene sus propios scripts: npm test y npm run coverage, y en web/ además npm run test:e2e.
- Para que Supertest pueda usar la API sin abrir un puerto, se separa api/app.js en dos archivos: app.js crea y exporta la aplicación de Express (export default app) y server.js conecta la base de datos, crea el administrador inicial y llama a listen(). Los scripts dev y start pasan a ejecutar server.js. No se cambia ninguna otra lógica.
- Las pruebas de la API no se conectan a la base de datos real: usan mongodb-memory-server, y JWT_SECRET y JWT_EXPIRA se definen en vitest.config.js (env).
- Cada prueba es independiente: la base de datos y localStorage se limpian antes de cada prueba y ninguna prueba depende del orden de ejecución.
- Las pruebas de la web y de Playwright usan los atributos data-test que ya tiene el proyecto (por ejemplo login-email, login-submit, user-row, users-search), no clases CSS ni textos de botones.
- Los nombres de las pruebas describen el caso y el resultado esperado, por ejemplo: "GET /api/usuarios con token de usuario responde 403".
- El .env, la API key de TestSprite y las carpetas node_modules/, coverage/, playwright-report/ y test-results/ no se suben al repositorio; sus resultados se entregan como capturas y archivos en evidencias/.

Estructura de carpetas:

```
app-usuarios/
├── api/
│   ├── app.js                  ← crea y exporta la app de Express
│   ├── server.js               ← conecta la BD y llama listen()
│   ├── controllers/ helpers/ middlewares/ models/ routes/ database/
│   ├── tests/
│   │   ├── setup.js            ← mongodb-memory-server y limpieza
│   │   ├── helpers.js          ← crearUsuario(), tokenAdmin(), tokenUsuario()
│   │   ├── unit/               ← middlewares, helpers y modelo
│   │   └── integration/        ← auth.test.js y usuarios.test.js (Supertest)
│   └── vitest.config.js
├── web/
```

```
│   ├── src/                    ← código existente
│   ├── tests/unit/             ← validators, http, composables y componentes
│   ├── e2e/                    ← flujos con Playwright
│   └── vitest.config.js / playwright.config.js
├── evidencias/
│   ├── coverage/  playwright/  testsprite/
├── INFORME_PRUEBAS.md
└── README.md
```

1. Configuración del entorno de pruebas. Instala las dependencias de desarrollo, separa app.js de server.js y define los scripts de cada proyecto.

En api/:

```
npm i -D vitest @vitest/coverage-v8 supertest mongodb-memory-server
```

En web/:

```
npm i -D vitest @vitest/coverage-v8 @vue/test-utils jsdom
npm init playwright@latest
```

La primera vez que se ejecuta, mongodb-memory-server descarga un binario de MongoDB (alrededor de 100 MB): hacerlo con buena conexión antes de la clase.

Scripts obligatorios en cada package.json:

Script	Comando	Proyecto
dev / start	node --watch server.js / node server.js	api
test	vitest run	api y web
test:watch	vitest	api y web
coverage	vitest run --coverage	api y web
test:e2e	playwright test	web

En web/vitest.config.js se reutiliza la configuración de Vite (plugin de Vue y alias @) y se agrega environment: "jsdom". En api/vitest.config.js se agrega setupFiles: ["tests/setup.js"].

2. Pruebas de la API con Vitest y Supertest. Verifica los endpoints de la App de usuarios sin levantar el servidor y con una base de datos en memoria.

Ejemplo de una prueba de integración:

```
import { describe, it, expect } from "vitest";
import request from "supertest";
import app from "../../app.js";
import { tokenUsuario } from "../helpers.js";

describe("GET /api/usuarios", () => {
  it("con token de usuario responde 403", async () => {
    const token = await tokenUsuario();
```

```
    const res = await request(app).get("/api/usuarios").set("x-token", token);
    expect(res.status).toBe(403);
    expect(res.body.error).toBe("No tiene permisos para esto");
  });
});
```

Casos mínimos de autenticación (/api/auth):

Petición	Caso	Esperado
POST /registro	Datos válidos: devuelve token, rol "usuario" aunque se envíe rol "admin", y no incluye password	201
POST /registro	Email ya registrado (errores con path "email")	400
POST /registro	Contraseña sin mayúscula o con menos de 8 caracteres	400
POST /login	Credenciales correctas: devuelve token y usuario con uid	200
POST /login	Contraseña incorrecta y email inexistente: mismo mensaje “Credenciales inválidas”	401
POST /login	Usuario desactivado (estado: false)	401
GET /perfil	Sin token, con token inválido y con token válido	401 / 401 / 200
PUT /password	Contraseña actual incorrecta, y nueva igual a la actual	400
PUT /password	Cambio correcto: luego el login funciona con la nueva contraseña	200

Casos mínimos del CRUD de usuarios (/api/usuarios):

Petición	Caso	Esperado
GET /	Sin token / con token de rol usuario	401 / 403
GET /	Admin: total, pagina, limite, totalPaginas y usuarios; solo activos	200
GET /?limite=51	Límite fuera de rango (máximo 50)	400
GET /?buscar=ana&rol=usuario	Búsqueda por nombre o email combinada con el filtro de rol	200
GET /:id	Id mal formado (abc) / id válido que no existe	400 / 404
POST /	Admin crea usuario: la contraseña queda hasheada en la base de datos	201
PUT /:id	Admin intenta cambiar su propio rol	400
PUT /:id	Password vacía: no se cambia y el login sigue funcionando	200
DELETE /:id	Admin intenta eliminarse a sí mismo	400
DELETE /:id	Borrado lógico: deja de aparecer en el listado y ya no puede iniciar sesión	200

Otros casos:

- GET /api/health responde 200 con { status: "ok" } y una ruta inexistente responde 404 en JSON.
- Un body con JSON mal formado responde 400 con “El cuerpo de la petición no es un JSON válido”.
- Pruebas unitarias sin base de datos de validarRol (500 sin req.usuario, 403 con rol no permitido, next() con rol permitido), esRolValido, generarJWT (el token contiene uid y rol) y toJSON del modelo (no devuelve password ni _id).

El sistema de pruebas debe además:

- Configurar tests/setup.js para iniciar mongodb-memory-server y conectar Mongoose antes de todas las pruebas, vaciar la colección de usuarios antes de cada prueba y desconectar al final.
- Crear en tests/helpers.js funciones para crear usuarios y obtener un token de admin y otro de usuario.
- Usar vi.fn() para simular req, res y next en las pruebas unitarias de los middlewares.
- Tener como mínimo 30 pruebas en la API, todas pasando.

3. Pruebas unitarias y de componentes en la web (Vitest + Vue Test Utils). Verifica la lógica del cliente sin depender de la API: las llamadas de red se simulan.

Ejemplo de prueba de componente:

```
import { mount } from "@vue/test-utils";
import PaginationBar from "@/components/PaginationBar.vue";

it("en la página 1 deshabilita Anterior y emite change", async () => {
  const wrapper = mount(PaginationBar, {
    props: { page: 1, totalPages: 3, total: 20, limit: 8 }
  });
  const $ = (id) => wrapper.find(`[data-test=${id}]`);
  expect($("pagination-info").text()).toContain("1–8 de 20");
  expect($("pagination-prev").attributes("disabled")).toBeDefined();
  await $("pagination-next").trigger("click");
  expect(wrapper.emitted("change")[0]).toEqual([2]);
});
```

Casos mínimos:

Archivo	Caso
utils/validators.js	validateName, validateEmail y validatePasswordConfirmation: vacío, inválido y válido
utils/validators.js	validatePassword: un caso por cada regla (longitud, mayúscula, minúscula y número) e isValid
services/http.js	Envía el header x-token si hay token guardado y serializa el body como JSON (fetch simulado con vi.stubGlobal)
services/http.js	Convierte { errores: [...] } en ApiError con details por campo, y { error } en ApiError con ese mensaje

services/http.js	Si fetch falla lanza ApiError con status 0; con 401 y token guardado llama al manejador de setUnauthorizedHandler
composables/useToast.js	success y error agregan un toast y se eliminan solos al pasar el tiempo (vi.useFakeTimers)
components/FormField.vue	Muestra el error con data-test {testId}-error y el botón Mostrar/Ocultar cambia el tipo del input
components/PaginationBar.vue	Texto “Mostrando X–Y de Z”, botones deshabilitados en los límites y evento change
components/ConfirmDialog.vue	Emite confirm y cancel con los botones confirm-accept y confirm-cancel
views/LoginView.vue	Con email inválido muestra el error sin llamar a la API; con error del servidor muestra el alert (authApi simulado con vi.mock)

El sistema de pruebas debe además:

- Limpiar localStorage y restaurar los mocks (vi.restoreAllMocks) antes de cada prueba.
- Para las vistas que usan Vue Router, montar con un router de prueba (createRouter con createMemoryHistory) o usar stubs.
- Tener como mínimo 30 pruebas en la web, todas pasando.

4. Pruebas end-to-end con Playwright. Verifica flujos completos en un navegador real, con la API y la web funcionando juntas.

Configuración en playwright.config.js:

```
use: { baseURL: "http://localhost:5173", testIdAttribute: "data-test",
       screenshot: "only-on-failure", trace: "on-first-retry" },
webServer: [
  { command: "npm run seed && npm run dev", cwd: "../api", port: 3000,
    env: { MONGODB_CNX: "mongodb://127.0.0.1:27017/app-usuarios-e2e" } },
  { command: "npm run dev", port: 5173 }
]
```

npm run seed borra y vuelve a cargar los usuarios de prueba, por eso Playwright usa una base de datos aparte (app-usuarios-e2e) y nunca la de desarrollo. Cuentas de prueba del seed: admin@demo.com / Admin123! y ana@demo.com / Usuario123.

Ejemplo de prueba E2E:

```
import { test, expect } from "@playwright/test";

test("el admin inicia sesión y ve la lista de usuarios", async ({ page }) => {
  await page.goto("/login");
  await page.getByTestId("login-email").fill("admin@demo.com");
  await page.getByTestId("login-password").fill("Admin123!");
  await page.getByTestId("login-submit").click();
  await expect(page).toHaveURL(/\/usuarios/);
  await expect(page.getByTestId("user-row").first()).toBeVisible();
```

```
});
```

Flujos mínimos:

- Login con credenciales incorrectas: aparece el alert con “Credenciales inválidas” y se sigue en /login.
- Registro de una cuenta nueva: entra a /perfil y el rol mostrado es Usuario.
- Un usuario con rol usuario que abre /usuarios es redirigido a /perfil; sin sesión, /usuarios redirige a /login.
- El admin crea un usuario, lo encuentra con el buscador (users-search) y lo edita.
- El admin elimina un usuario con el diálogo de confirmación y ese usuario ya no puede iniciar sesión.
- Filtro por rol y paginación: el texto de pagination-info cambia al ir a la página siguiente.
- El usuario cambia su contraseña desde el perfil, cierra sesión (logout) e inicia sesión con la nueva.
- Una ruta que no existe muestra la página de “no encontrada”.

El sistema de pruebas debe además:

- Ejecutar las pruebas en Chromium y Firefox.
- Reutilizar el inicio de sesión del admin con storageState o con una función auxiliar, en lugar de repetir el login en cada prueba.
- Generar el reporte HTML (npx playwright show-report) y tomar capturas como evidencia.

5. Reporte de cobertura (coverage). Mide qué parte del código ejecutan las pruebas de la API y de la web.

Configuración en api/vitest.config.js:

```
test: {
  setupFiles: ["tests/setup.js"],
  coverage: {
    provider: "v8",
    reporter: ["text", "html", "json-summary"],
    include: ["app.js", "controllers/**", "middlewares/**",
              "helpers/**", "models/**", "routes/**"],
    thresholds: { lines: 80, functions: 80, statements: 80, branches: 70 }
  }
}
```

En web/ se incluye la carpeta src y se excluye src/main.js. Playwright no suma a este reporte: la cobertura se mide solo con las pruebas de Vitest.

Umbrales mínimos (si no se cumplen, npm run coverage debe fallar):

Métrica	Mínimo	Qué mide
Lines (líneas)	80%	Líneas de código ejecutadas por al menos una prueba
Statements (sentencias)	80%	Instrucciones ejecutadas
Functions (funciones)	80%	Funciones llamadas al menos una vez
Branches (ramas)	70%	Caminos de if, else, ternarios y operadores lógicos recorridos

El sistema debe además:

- Ejecutar npm run coverage en api/ y en web/ y tomar captura del resumen que aparece en la terminal.
- Abrir coverage/index.html, tomar captura de la tabla general y de un archivo con líneas sin cubrir (en rojo), y explicar en el informe por qué no estaban cubiertas y qué prueba se agregó.
- Copiar coverage/coverage-summary.json de cada proyecto a evidencias/coverage/ (api-coverage-summary.json y web-coverage-summary.json).

6. Pruebas con TestSprite. Usa el agente de pruebas con IA de TestSprite para generar y ejecutar casos de prueba adicionales sobre la API y la web, y compáralos con las pruebas escritas a mano.

Configuración del servidor MCP en el IDE (VS Code, Cursor o Claude Code):

```
{
  "mcpServers": {
    "TestSprite": {
      "command": "npx",
      "args": ["@testsprite/testsprite-mcp@latest"],
      "env": { "API_KEY": "tu-api-key" }
    }
  }
}
```

- Crear una cuenta gratuita en testsprite.com, generar la API key y configurarla en el IDE (nunca en el repositorio).
- Escribir un PRD.md corto con los requisitos de la App de usuarios: endpoints, roles, reglas de contraseña, borrado lógico, reglas de negocio (un admin no puede cambiar su propio rol ni eliminarse) y cuentas de prueba del seed.
- Levantar el proyecto antes de iniciar: en api/ npm run seed y npm run dev (puerto 3000), y en web/ npm run dev (puerto 5173).
- Pedir al asistente del IDE: “Help me test this project with TestSprite”. Hacer una ejecución de tipo backend para la API (puerto 3000) y otra de tipo frontend para la web (puerto 5173), cargando el PRD.md en ambas.
- Revisar y ajustar el plan de pruebas que propone TestSprite antes de ejecutarlo.

- Analizar cada caso que falle: decidir si es un error real del código, un requisito mal interpretado o un problema del entorno; corregir lo que sea un error real y volver a ejecutar TestSprite.

TestSprite crea la carpeta testsprite_tests/ con el plan de pruebas, los casos generados y los reportes TestSprite_MCP_Test_Report.md y .html. Esa carpeta se copia a evidencias/testsprite/api y evidencias/testsprite/web (sin archivos que contengan la API key).

Tabla de análisis que se incluye en el informe por cada caso fallido (ejemplo):

Caso	Descripción	Resultado	Causa	Decisión
TC007	El admin cambia su propio rol a usuario	Falló: esperaba 200	Requisito mal interpretado: la regla exige 400	Se aclaró la regla en el PRD.md; no se cambió el código
…	…	…	…	…

7. Informe de pruebas. En INFORME_PRUEBAS.md se resume todo el trabajo:

- Tabla resumen por proyecto y tipo de prueba: herramienta, número de pruebas, cuántas pasan, cuántas fallan y porcentaje de cobertura.
- Lista de errores o comportamientos dudosos encontrados gracias a las pruebas y qué se hizo con cada uno (mínimo 2).
- Tabla de análisis de TestSprite (punto 6) y resultado de la segunda ejecución.
- Comparación de mínimo un párrafo: qué encontró TestSprite que no cubrían las pruebas escritas a mano, qué no detectó, y en qué casos conviene cada enfoque.
- Las capturas de pantalla enlazadas desde la carpeta evidencias/.

Evidencias a cargar

Evidencia	Formato	Ubicación
Ejecución de npm test en api y web (todas las pruebas pasando)	Captura PNG	evidencias/coverage/
Resumen de cobertura en la terminal de api y web	Captura PNG	evidencias/coverage/
Reporte HTML de cobertura (tabla general y archivo con líneas sin cubrir)	Captura PNG + coverage-summary.json	evidencias/coverage/
Reporte HTML de Playwright (Chromium y Firefox)	Captura PNG	evidencias/playwright/
Plan, casos y reporte de TestSprite de la API	Carpeta testsprite_tests + .md / .html	evidencias/testsprite/api/
Plan, casos y reporte de TestSprite de la web	Carpeta testsprite_tests + .md / .html	evidencias/testsprite/web/

Panel de TestSprite con los resultados de ambas ejecuciones	Captura PNG	evidencias/testsprite/
Documento de requisitos usado en TestSprite	Markdown	PRD.md
Informe de pruebas	Markdown	INFORME_PRUEBAS.md

Lista de verificación antes de entregar

- npm install y npm test funcionan en otro computador en api/ y en web/ sin crear el .env ni tener MongoDB instalado.
- api/app.js exporta la app sin llamar a listen() y npm run dev sigue funcionando con server.js.
- La API tiene al menos 30 pruebas y la web al menos 30, todas pasando.
- npm run coverage cumple los umbrales en ambos proyectos (80% líneas, funciones y sentencias; 70% ramas).
- npm run test:e2e pasa en Chromium y Firefox usando la base de datos app-usuarios-e2e.
- Las pruebas no dependen del orden de ejecución ni de datos que queden de otra prueba.
- Están las evidencias de TestSprite de la API y de la web, con la tabla de análisis de fallos.
- No hay .env, API keys, node_modules, coverage/ ni playwright-report/ en el repositorio.
- INFORME_PRUEBAS.md tiene el resumen, los hallazgos y la comparación con TestSprite.

Criterios de evaluación

Criterio	**Peso**
Configuración: dependencias, scripts, separación app.js / server.js, base de datos en memoria y .gitignore	10%
Pruebas de la API: auth, CRUD, roles (401 / 403), reglas de negocio y pruebas unitarias	20%
Pruebas de la web: validators, http, composables y componentes	15%
Pruebas end-to-end con Playwright (API y web juntas)	15%
Cobertura: umbrales cumplidos y análisis de líneas sin cubrir	15%
TestSprite: PRD, ejecución en API y web, análisis de fallos y segunda ejecución	15%
Informe de pruebas y organización de evidencias	10%

Penalizaciones: −20% si el .env o la API key de TestSprite están en el repositorio; −15% si se modifican o eliminan pruebas solo para que pasen, o si se usan .skip / .only en la entrega; −10% si las pruebas de Vitest se conectan a una base de datos real. Bonus (hasta +10%): ejecutar las pruebas en GitHub Actions en cada push, pruebas de accesibilidad con @axe-core/playwright, pruebas de las guardas del router o cobertura superior al 90% en ambos proyectos.



PRODUCTO(S) ENTREGABLE(S)

Enlace del repositorio público en GitHub con la solución del **Taller de Pruebas de Software**, nombrado Taller_Pruebas_App_Usuarios_nombre_apellido_del_aprendiz, que contenga las carpetas api/ y web/ con sus pruebas, la carpeta evidencias/ con los reportes de cobertura, Playwright y TestSprite, los archivos PRD.md e INFORME_PRUEBAS.md y el README.md (cómo instalar, cómo ejecutar cada tipo de prueba e integrantes).

FORMA DE ENTREGA

El enlace del repositorio y las capturas de evidencia deberán subirse a Classroom en la fecha indicada por el instructor. Al inicio de cada clase se revisará el commit del avance anterior.